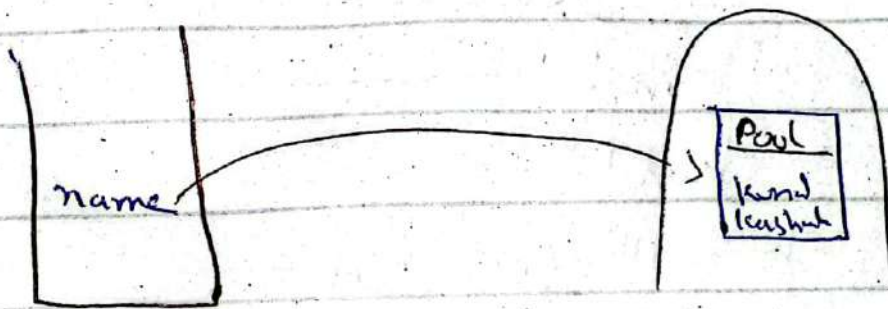


Strings & String Builder

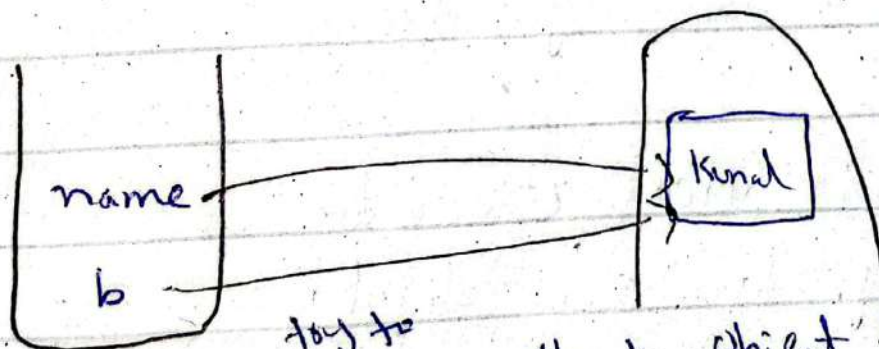
Strings →

String name = "Kunal Kushwaha"

class reference variable object
datatype



String pool:- is a separate memory structure inside the heap.



what if i change the b object from Kunal to Rahul?
we can't do b/c strings are immutable.

You can't modify an object (immutability)

Comparison of Strings:

① == method

↳
Comparator

reference variable
a → "kunal"
b → "kunal"
object

then $a == b$ will give false.

a → kunal

b →

then $a == b$ will give true.

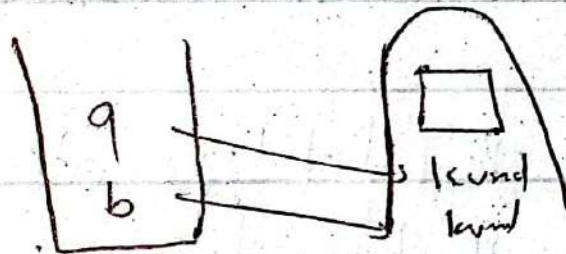
→ checks if reference variable are pointing to same object.

// How to create different objects of same value:-

```
String a = new String ("kunal");  
String b = new String ("kunal");
```

// Creating these values outside the pool but in heap

Internally,



(2) When you only need to check value, use equals method.

We can use ⁱⁿ complex expression + operator
but atleast one of the object should
be String.

Ex: String a = new Integer(56) + new ArrayList
<>(); // Error b/c no string included

String b = new Integer(56) + " " +
new ArrayList <>();

∴ if one of the datatype string, the
answer will be String.

String

```
String series = "";  
for (int i=0; i<26; i++) {  
    char ch = (char) ("a" + i);  
    series = series + ch;  
};
```

S.o.u.t (Series);

Internally:

- " $a + b = ab$
- " $ab + c = abc$
- " $abc + d = abcd$
- " $abcd + e = abcde$

} every time the object is changing (it is not added a new character to string but change the object everytime) b/c

Strings are immutable, so in result, a lot of memory wastage reducing efficiency.

Time Complexity: $O(N^2)$

So, is there is any other data type
so it just added new character to
already existed object/string but not
creating new object everytime so,
the answer is yes, It is
StringBuilder.

String Pool (why strings are immutable)?

Java stores String values in a shared memory
pool so if multiple variables have same
value, they point to one object instead
of creating duplicates. This is why
strings are immutable. If you could
change one variable value so it would
silently change every variable sharing
that value.